

Mohammed Yaacoub Abou Chlih 21138726

Matty Luriz 23206450

Github repository: <https://github.com/ExplosiveSalad/Quality-Assured-Vision.git>

Task 1: Project Completion and Final Implementation

Requirements refinement (not scope change)

- NFR-09's cold-start threshold was changed from a placeholder of <2000ms to <500ms once real data (~75-100ms typical cold-start) showed the original threshold was too lenient.

```
//NFR-09: cold-start (model load + first inference) latency, tracked separately
if (coldStartLatencyMs.HasValue)
{
    report.NfrResults.Add(new NfrCheckResult
    {
        RequirementId = "NFR-09",
        Description = "Cold-start latency (model load + first inference)",
        Passed = coldStartLatencyMs.Value < 500.0, //not placeholder anymore, data has been analysed
        ActualValue = $"{coldStartLatencyMs.Value:F2}ms",
        Threshold = "<500ms"
    });
}
```

- NFR-02's threshold (≥ 15 FPS for 95% of frames) was implemented and tested against real data, revealing the system always achieves ~10-12 FPS under baseline CPU-only inference which is an NFR failure, not an implementation gap/ This brings the question of whether 15 FPS is a realistic target for CPU-bound embedded-simulated inference, which we will revisit during quality metrics and release evaluation (Task 8) rather than treat it as a defect to silently fix by lowering the threshold.

The screenshot shows the Visual Studio Test Explorer interface. On the left, a tree view lists various tests under 'Test Explorer'. The 'Test run finished: 10 Tests (10 Passed, 0 Failed, 0 Skipped) run in 6.4 sec' message is visible. The 'Test Detail Summary' pane on the right shows the results for the test 'Generate_WithLargerFrameBatch_EvaluatesNfr02FrameRateCompliance'. The 'Standard Output' section displays a list of frames with their FPS values: Frame 1: 89.25ms (11.2 FPS), Frame 2: 89.92ms (11.1 FPS), Frame 3: 85.48ms (11.7 FPS), Frame 4: 87.58ms (11.4 FPS), Frame 5: 88.81ms (11.4 FPS), Frame 6: 91.21ms (11.0 FPS), Frame 7: 85.96ms (11.6 FPS), Frame 8: 89.23ms (11.2 FPS), Frame 9: 84.96ms (11.8 FPS), Frame 10: 81.86ms (12.3 FPS), Frame 11: 98.89ms (11.1 FPS), Frame 12: 85.76ms (11.7 FPS), Frame 13: 96.37ms (10.4 FPS), Frame 14: 83.21ms (12.0 FPS), Frame 15: 98.71ms (11.0 FPS), Frame 16: 81.18ms (12.3 FPS), Frame 17: 93.27ms (10.7 FPS), Frame 18: 88.86ms (11.4 FPS), Frame 19: 98.53ms (11.0 FPS), Frame 20: 98.22ms (10.2 FPS). The summary at the bottom states: 'NFR-02: Failed - At least 95% of frames sustain minimum 15 FPS (actual: 0.0% of frames >= 15 FPS, threshold: >=95% of frames >=15 FPS)'.

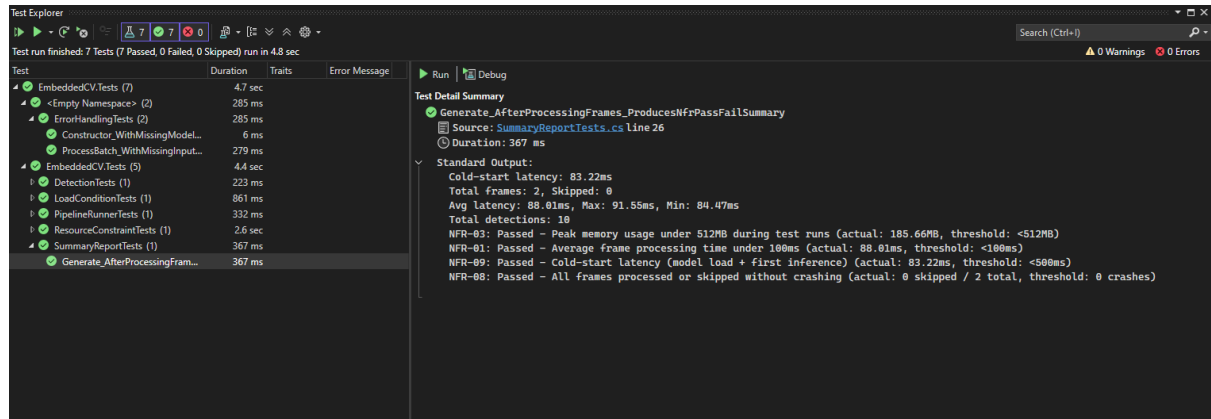
Mohammed Yaacoub Abou Chlih 21138726

Matty Luriz 23206450

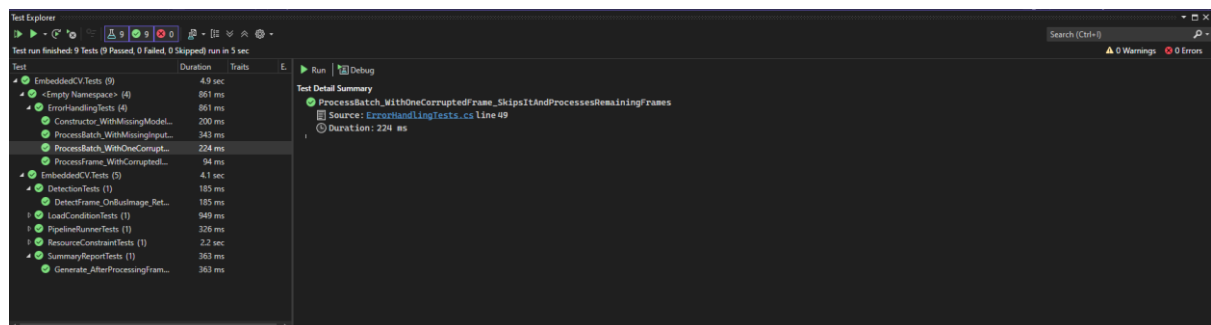
Github repository: <https://github.com/ExplosiveSalad/Quality-Assured-Vision.git>

Implementation additions (filling gaps seen in Assessment 1 feedback)

- NFR-03 (memory usage) previously had a defined threshold with no implemented measurement; this has been implemented using process working set memory sampling and is now actively tested



- NFR-08 (error handling) previously only verified that no unhandled exception occurred during a run; this has been changed with four dedicated test cases validating each specific error condition individually (missing model, corrupted frame, invalid input path), addressing the “false pass” risk seen in Assessment 1 feedback. Under ErrorHandlingTests are the four tests, covering Missing model (FR-09), Missing/invalid input path (FR-11), Single corrupted frame isolated (FR-10), and corrupted frame within a batch, confirming continuation (FR-10).



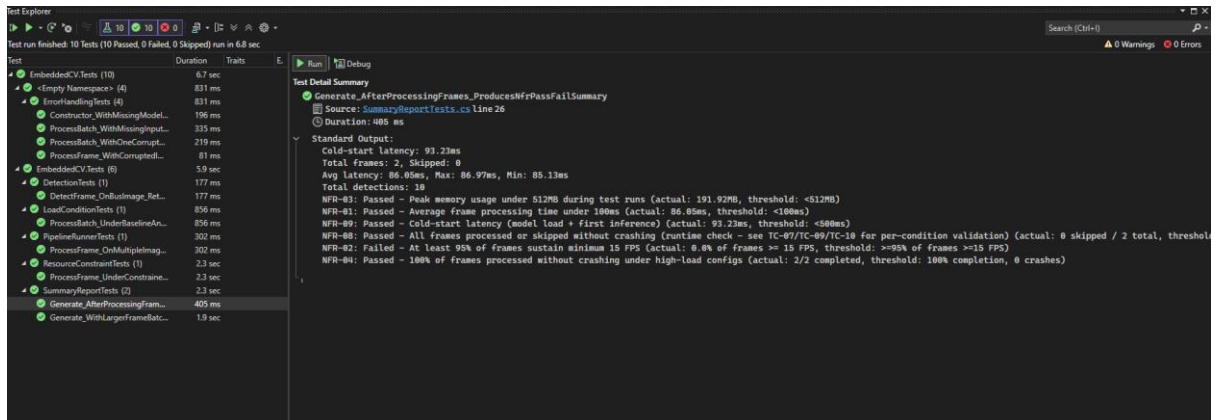
Mohammed Yaacoub Abou Chlih 21138726

Matty Luriz 23206450

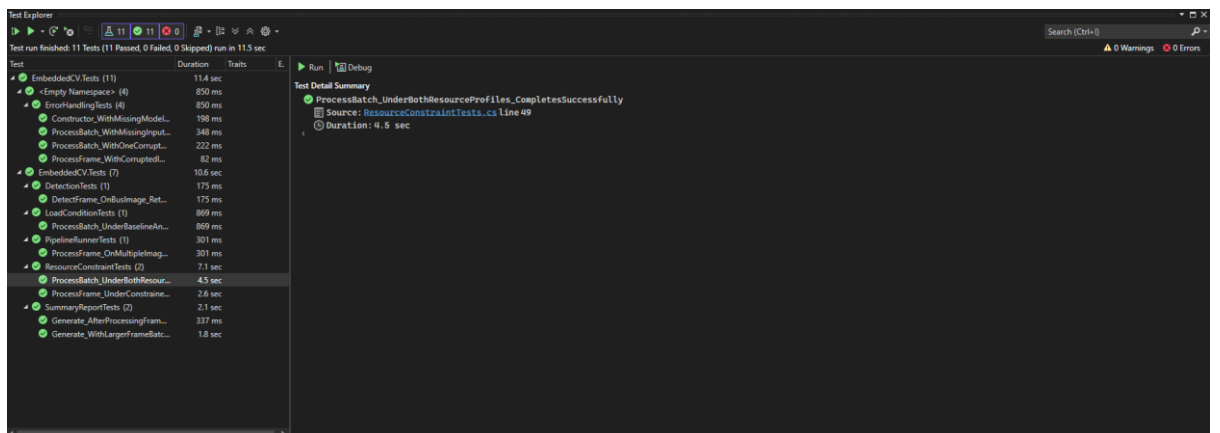
Github repository: <https://github.com/ExplosiveSalad/Quality-Assured-Vision.git>

- NFR-04, NFR-06, and NFR-07 previously untested, now have dedicated automated test coverage confirming reliability under load, consistency across repeated runs, and portability across hardware profiles.

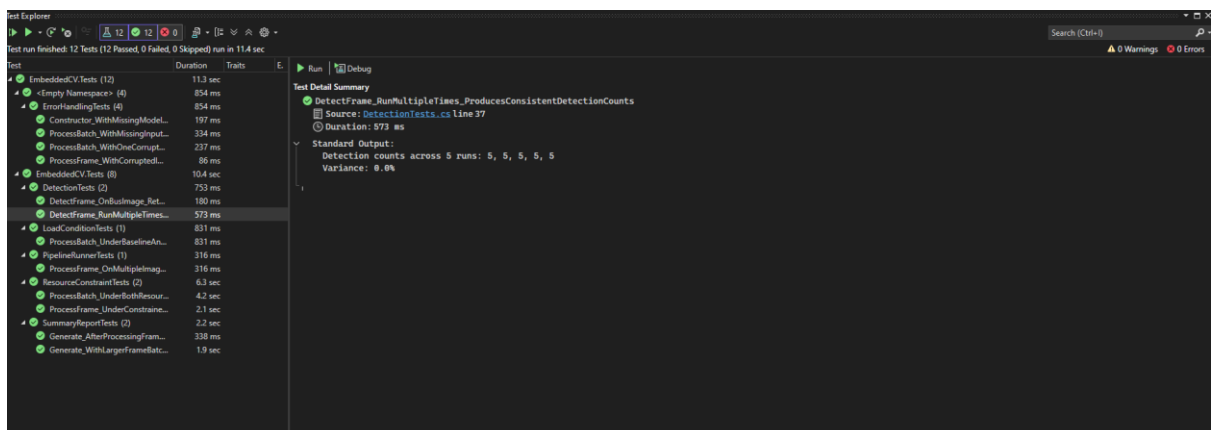
NFR-04



NFR-06



NFR-07

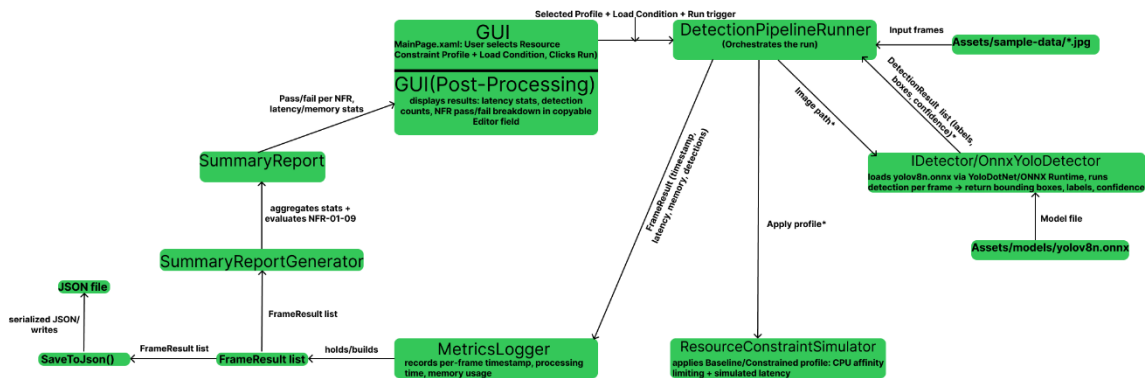


Mohammed Yaacoub Abou Chlih 21138726
Matty Luriz 23206450
Github repository: <https://github.com/ExplosiveSalad/Quality-Assured-Vision.git>

Architectural or technological changes

The core architecture (GUI → DetectionPipelineRunner → Detector/ResourceConstraintSimulator/MetricsLogging → SummaryReportGenerator) and the stack (C#/.NET 9, MAUI Windows, YoloDotNet/ONNX Runtime, MSTest) are unchanged from Assessment 1. Development since then is focused on finishing test coverage and measurement gaps seen in feedback, not restructuring the system.

Architecture/Data Flow diagram



*: For the setup of those marked, Image path and DetectionResult list are per-frame, repeated calls
But Apply profile is a one-time setup call (before the per-frame loop begins)

GUI(Profile: Baseline, Load condition: Baseline)

Home

Embedded CV QA Dashboard

Resource Constraint Profile

Select profile

Baseline

Load Condition

Select load condition

Baseline

Run Test

Run complete.

Results

Profile: Baseline | Load: Baseline

Frames: 2 (Skipped: 0)
Avg latency: 87.84ms
Max latency: 88.16ms
Min latency: 87.52ms
Total detections: 10

NFR Results:

NFR-03: PASS — Peak memory usage under 512MB during test runs (actual: 300.27MB, threshold: <512MB)
NFR-01: PASS — Average frame processing time under 100ms (actual: 87.84ms, threshold: <100ms)
NFR-08: PASS — All frames processed or skipped without crashing (runtime check - see TC-07/TC-09/TC-10 for per-condition validation) (actual: 0 skipped / 2 total, threshold: 0 crashes)
NFR-02: FAIL — At least 95% of frames sustain minimum 15 FPS (actual: 0.0% of frames >= 15 FPS, threshold: >=95% of frames >= 15 FPS)
NFR-04: PASS — 100% of frames processed without crashing under high-load configs (actual: 2/2 completed, threshold: 100% completion, 0 crashes)

Mohammed Yaacoub Abou Chlih 21138726
Matty Luriz 23206450
Github repository: <https://github.com/ExplosiveSalad/Quality-Assured-Vision.git>

GUI(Profile: Baseline, Load condition: High load)

Home

Embedded CV QA Dashboard

Resource Constraint Profile

Select profile

Baseline

Load Condition

Select load condition

High Load

Run Test

Run complete.

Results

Profile: Baseline | Load: HighLoad

Frames: 2 (Skipped: 0)
Avg latency: 136.57ms
Max latency: 164.49ms
Min latency: 108.65ms
Total detections: 10

NFR Results:
NFR-03: PASS — Peak memory usage under 512MB during test runs (actual: 312.88MB, threshold: <512MB)
NFR-01: FAIL — Average frame processing time under 100ms (actual: 136.57ms, threshold: <100ms)
NFR-08: PASS — All frames processed or skipped without crashing (runtime check - see TC-07/TC-09/TC-10 for per-condition validation) (actual: 0 skipped / 2 total, threshold: 0 crashes)
NFR-02: FAIL — At least 95% of frames sustain minimum 15 FPS (actual: 0.0% of frames >= 15 FPS, threshold: >=95% of frames >= 15 FPS)
NFR-04: PASS — 100% of frames processed without crashing under high-load configs (actual: 2/2 completed, threshold: 100% completion, 0 crashes)

GUI(Profile: Constrained, Load condition: Baseline)

Home

Embedded CV QA Dashboard

Resource Constraint Profile

Select profile

Constrained (Embedded-like)

Load Condition

Select load condition

Baseline

Run Test

Run complete.

Results

Profile: Constrained (Embedded-like) | Load: Baseline

Frames: 2 (Skipped: 0)
Avg latency: 2673.44ms
Max latency: 2980.70ms
Min latency: 2366.18ms
Total detections: 10

NFR Results:
NFR-03: PASS — Peak memory usage under 512MB during test runs (actual: 318.76MB, threshold: <512MB)
NFR-01: FAIL — Average frame processing time under 100ms (actual: 2673.44ms, threshold: <100ms)
NFR-08: PASS — All frames processed or skipped without crashing (runtime check - see TC-07/TC-09/TC-10 for per-condition validation) (actual: 0 skipped / 2 total, threshold: 0 crashes)
NFR-02: FAIL — At least 95% of frames sustain minimum 15 FPS (actual: 0.0% of frames >= 15 FPS, threshold: >=95% of frames >= 15 FPS)
NFR-04: PASS — 100% of frames processed without crashing under high-load configs (actual: 2/2 completed, threshold: 100% completion, 0 crashes)

GUI(Profile: Constrained, Load condition: High load)

Home

Embedded CV QA Dashboard

Resource Constraint Profile

Select profile

Constrained (Embedded-like)

Load Condition

Select load condition

High Load

Run Test

Run complete.

Results

Profile: Constrained (Embedded-like) | Load: HighLoad

Frames: 2 (Skipped: 0)
Avg latency: 3917.87ms
Max latency: 4402.50ms
Min latency: 3433.25ms
Total detections: 10

NFR Results:
NFR-03: PASS — Peak memory usage under 512MB during test runs (actual: 321.32MB, threshold: <512MB)
NFR-01: FAIL — Average frame processing time under 100ms (actual: 3917.87ms, threshold: <100ms)
NFR-08: PASS — All frames processed or skipped without crashing (runtime check - see TC-07/TC-09/TC-10 for per-condition validation) (actual: 0 skipped / 2 total, threshold: 0 crashes)
NFR-02: FAIL — At least 95% of frames sustain minimum 15 FPS (actual: 0.0% of frames >= 15 FPS, threshold: >=95% of frames >= 15 FPS)
NFR-04: PASS — 100% of frames processed without crashing under high-load configs (actual: 2/2 completed, threshold: 100% completion, 0 crashes)

Mohammed Yaacoub Abou Chlih 21138726

Matty Luriz 23206450

Github repository: <https://github.com/ExplosiveSalad/Quality-Assured-Vision.git>

Deferred, not implemented

A feature, considered previously, which would allow users to upload their own dataset for testing (rather than the fixed sample set) was evaluated and deliberated on by Me (Yaacoub) and Matthew, we concluded that it doesn't map to any required Assessment 2 task and was judged not a good use of our time. This is documented as a future recommendation (Task 10) rather than abandoned.

NFR-05 (configurability) hasn't been automated, since it's fundamentally a GUI usability requirement rather than backend logic suited to unit testing. It'll be validated as a manual usability test case under Task 7 (Quality Testing), supported by GUI screenshots.

Task 2: Requirements Verification and Traceability

- Review and finalise the functional and non-functional requirements from Assessment 1.
- Update the Requirements Traceability Matrix.
- Map requirements to implemented features, test cases, test results, and defects where applicable.
- Identify any requirements that are partially implemented, unverified, or out of scope, and justify them.

Review and finalisation of requirements

The baseline is made up of the 11 functional requirements (FR-01 to FR-11) and the 9 non-functional requirements (NFR-01 to NFR-09) (including the updated and testable non-functional requirements, the error handling requirements (FR-09 to FR-11), and the Assessment 1 introduction of NFR-08 and NFR-09), which have remained constant since Assessment 1, with no additions or deletions made. Only the testing of these requirements has changed; NFR-03 is checked by a real memory measurement, NFR-08 by a different test for each error condition, and NFR-02, NFR-04, NFR-06, and NFR-07 are now tested with automated tests, whereas at mid-point, none of these four requirements had been tested.

The numbering of test cases here replaces the nine-test case numbering used in the traceability matrix for Assessment 1. The 15 FPS mark for NFR-02 stands, with the same failure in the current system; this is evaluated in Task 8 rather than

Mohammed Yaacoub Abou Chlih 21138726

Matty Luriz 23206450

Github repository: <https://github.com/ExplosiveSalad/Quality-Assured-Vision.git>

reducing the bar. Comparison of the baseline with the final system reveals three discrepancies between the code and the requirements.

FR-06 and NFR-07 mention CPU and memory usage limits, but the code only restricts the CPU to a single core under the constrained profile and adds a fixed delay per frame, with no memory limit; similarly, though FR-07 mentions detection accuracy, the summary report does not include an accuracy measure.

NFR-01 and NFR-02 are functionally overlapping, because at 15 FPS (a frame time of 66.7ms) any frame satisfying NFR-02 will also satisfy NFR-01's 100ms barrier. This is, however, not remedied (similarly to the above two points), to keep the requirements as originally written in Assessment 1.

Requirements Traceability Matrix

Test case IDs refer to the final test case table in Task 3. Defect IDs refer to the table that follows and are to be aligned with the defect register in Task 6.

ID	Requirement	Implemented in	Test case(s)	Result	Defect
FR-01	Load and run a pre-trained detection model on input data	OnnxYoloDetector (constructor)	TC-01	Pass	None
FR-02	Detect and classify objects in each frame	OnnxYoloDetector.DetectFrame	TC-01, TC-03	Pass (accuracy not verified)	None
FR-03	Output bounding boxes, labels and confidence scores	DetectionResult	TC-01, TC-03	Pass	None
FR-04	Process pre-recorded input as a simulated feed	DetectionPipelineRunner.ProcessBatch	TC-04, TC-11	Pass	None
FR-05	Log each frame's results and timestamp	MetricsLogger, FrameResult, SaveToJson	TC-04, TC-05	Pass	None
FR-06	Configurable simulated resource constraint before a run	ResourceConstraintProfile, ResourceConstraintSimulator, GUI profile picker	TC-06, TC-07	Partial (no memory cap)	None

Mohammed Yaacoub Abou Chlih 21138726

Matty Luriz 23206450

Github repository: <https://github.com/ExplosiveSalad/Quality-Assured-Vision.git>

ID	Requirement	Implemented in	Test case(s)	Result	Defect
FR-07	Summary report of accuracy and performance metrics	SummaryReportGenerator, GUI results panel	TC-08, TC-09, TC-10	Partial (no accuracy metric)	DEF-05
FR-08	Select between at least two load conditions	LoadCondition, ProcessBatch, GUI load picker	TC-11	Pass	None
FR-09	Stop and log an error if the model cannot load	OnnxYoloDetector (constructor)	TC-12	Pass	None
FR-10	Skip a corrupted frame, log it, continue	DetectionPipelineRunner.ProcessFrames	TC-14	Pass	None
FR-11	Reject a run before execution if input is missing or invalid	DetectionPipelineRunner.ValidateInputPaths	TC-13, TC-15	Pass	None
NFR-01	Frame processing time under 100ms	SummaryReportGenerator (NFR-01 check)	TC-06, TC-08	Pass for Baseline/Baseline (87.84ms); exceeded in the other three configurations	None
NFR-02	At least 95% of frames sustain 15 FPS	SummaryReportGenerator (NFR-02 check)	TC-09	Fail (0.0% of frames; about 11 FPS average)	DEF-01
NFR-03	Peak memory under 512MB	FrameResult.MemoryUsedBytes, SummaryReportGenerator (NFR-03 check)	TC-08, TC-10 (see note)	Pass (about 186 to 192MB in test runs, 300 to 322MB in GUI runs)	DEF-03 (closed)
NFR-04	100% of frames complete under high load	SummaryReportGenerator (NFR-04 check), Parallel.For in ProcessBatch	TC-11	Pass (2/2 completed in all four configurations)	None
NFR-05	Configure and launch a run in 3 steps or fewer, without editing source	MainPage.xaml (two pickers and a run button)	Manual GUI test (Task 7)	Not automated	None

Mohammed Yaacoub Abou Chlih 21138726

Matty Luriz 23206450

Github repository: <https://github.com/ExplosiveSalad/Quality-Assured-Vision.git>

ID	Requirement	Implemented in	Test case(s)	Result	Defect
NFR-06	Results within 2% variance across 5 repeated runs	DetectionTests variance check	TC-02	Pass (0% variance)	None
NFR-07	Full run completes under at least 2 simulated hardware profiles	ResourceConstraintSimulator, Baseline and Constrained profiles	TC-07	Pass	None
NFR-08	All defined error conditions handled without crashing	ProcessFrames try/catch, ValidateInputPaths, detector constructor, NFR-08 report check	TC-12, TC-13, TC-14, TC-15	Pass	DEF-04 (closed), DEF-05
NFR-09	Cold-start latency under 500ms	SummaryReportGenerator (NFR-09 check)	TC-10	Pass	None

Defect Mapping

ID	Defect or finding	Related requirement and test	Status
DEF-01	System sustains about 10 to 12 FPS on CPU-only YOLOv8n inference against a 15 FPS threshold.	NFR-02, TC-09	Open. Threshold review in Task 8.
DEF-02	Parallel test execution leaked the CPU affinity setting between tests, producing inconsistent latency results.	NFR-01, NFR-02	Closed. DoNotParallelize applied and affinity reset in test setup and cleanup.
DEF-03	NFR-03 had a defined threshold but no measurement in code.	NFR-03	Closed. Peak working set sampled per frame and checked in the report.

ID	Defect or finding	Related requirement and test	Status
DEF-04	NFR-08 report check passed unconditionally if the run returned.	NFR-08, TC-12 to TC-15	Closed. Per-condition tests added. The report check remains a runtime check only.
DEF-05	The NFR-08 description string shown in the GUI and test output cites TC-07, TC-09 and TC-10, which do not match the final test case numbering (TC-12 to TC-15).	FR-07, NFR-08	Open. Low severity.
DEF-06	MSTestSettings.cs requests method-level parallel execution while AssemblyInfo.cs sets DoNotParallelize. The effective behaviour has not been confirmed.	NFR-01, NFR-02 (timing reliability)	Open. Low severity.

- **FR-06 (partially implemented).** The constrained profile forces the process onto a single CPU core, and has a further 50ms of emulated latency per frame. No memory cap has been implemented, so the constraint (FR-06/NFR-07) is a CPU-only constraint. Memory is monitored and verified against the 512MB limit (NFR-03) but not enforced. This was deemed suitable because the research objective is to investigate non-functional behaviour under emulation, and the two profiles are sufficient to demonstrate NFR-07.
- **FR-07 (partially implemented) and detection accuracy (unverified).** The report summarises the system: latency, RAM, number of detections and if NFRs pass or fail. It does not include a measure of detection accuracy, and no detection comparison to manually labelled samples has been performed, as required by the acceptance criteria of FR-02 and FR-03. Because TC-01 and TC-03 only check whether detections are produced, not whether they are the correct detections, an accuracy value cannot be derived from them. Accuracy of the YOLOv8n model is beyond

the scope of this project (verification of non-functional behaviour) and so has been marked as unverified.

- **NFR-02 (implemented, not passing).** TC-09 tests the requirement correctly and fails for all runs (0.0% of frames greater than or equal to 15 FPS). The test itself is passing in MSTest since the test only asserts the number of frames, so the failure appears in the report output and not as a red test in Test Explorer. Recorded as DEF-01 and taken to Task 8, where the realism of 15 FPS for CPU-only inference is considered.
- **NFR-01 (validated only for the baseline run).** The acceptance criterion is met in the baseline run (average latency 87.84ms). The same check exceeds 100ms in Baseline with High Load (136.57ms), Constrained with Baseline load (2673.44ms) and Constrained with High Load (3917.87ms). The two constrained values include the injected 50ms delay and single core limit so are to be expected. The wording is also inconsistent across the requirement, acceptance criterion and implementation: the rewritten requirement specifies each frame under 100ms, the acceptance criterion specifies 95% of frames and the implementation checks the average latency.
- **NFR-05 (verified manually).** Configurability is a GUI usability requirement and is not amenable to automated testing. Verified using a manual test case in Task 7 supported by the GUI screenshots for all four profile and load combinations.
- **NFR-06 (partially verified).** TC-02 calculates how much the count of detections changes in five runs of a single image. The reworded requirement is referring to class labels and bounding boxes and the test does not test the variance in bounding boxes, so consistency is only verified to the extent that it covers the need.
- **Sample data coverage (limitation).** All tests employ two base images, repeated in volume. Inclusion of an option for users to provide their own data set was initially slated for a future Task 10 recommendation, as this functionality does not appear to translate into a required Assessment 2 task.
- **Security, accessibility and compatibility (out of scope).** No requirements for these were defined in Assessment 1. The system is local-only, with no attack surface outside of the device, no interface relevant to accessibility except for one dashboard page, and one target platform (.NET MAUI on Windows).

Mohammed Yaacoub Abou Chlih 21138726

Matty Luriz 23206450

Github repository: <https://github.com/ExplosiveSalad/Quality-Assured-Vision.git>

Task 3: Test Execution and Final Test Results

Execution Summary

Metric	Count
Planned	15
Executed	15
Passed	14
Failed	1 (TC-09)
Blocked	0
Not Run	0

Explanation of Failed Test and Corrective Action

TC-09 (NFR-02, frame rate compliance) is the only failed test case, and it fails by design rather than a defect. The test method itself runs properly and accurately measures real system behaviour. The system under test doesn't meet the 15 FPS threshold, consistently getting ~10-12 FPS on CPU-only YOLOv8n inference across a 20-frame sample. This is being treated as an actual, informative NFR failure rather than a bug to patch, since it demonstrates the real-world software quality problem this project sought out to investigate (Functional correctness does not guarantee real-time performance compliance). The corrective action under consideration is not a code fix, but a requirement-level discussion whether the 15 FPS threshold is realistic for CPU-only inference in an embedded-simulated context. It's to be resolved during quality metrics and release evaluation (Task 8). No regression risk exists from this result, since the failure shows measured hardware/software performance, not bad logic.

Test Case Table

ID	Test Case	Preconditions	Steps	Input	Expected Result	Requirements	Type	Priority	Status
TC-01	Detector returns valid detections on known sample image	Model + sample image available	1. Load detector 2. Call DetectFrame(bus.jpg)	bus.jpg	Returns list of detections with labels, confidence, bounding box	FR-01-04	Unit/Integration	High	Pass
TC-02	Detection results consistent across repeated runs	Model loaded, warmed up	1. Run DetectFrame 5x on same image	bus.jpg x5	Detection count variance $\leq 2\%$	NFR-06	Non-functional	Medium	Pass (0% variance)
TC-03	Detector returns empty list on blank image	Model loaded	1. Generate blank image 2. Run DetectFrame	Blank white image	Returns empty detection list, no error	FR-02, FR-03	Boundary	Medium	Pass
TC-04	Full pipeline logs results and timing for multiple frames	Model + 2 sample images available	1. Process bus.jpg, frame 1 2. Process zidane.jpg, frame 2	bus.jpg, zidane.jpg	2 FrameResults logged with valid detections, timestamps, latency	FR-04, FR-05	Integration	High	Pass
TC-05	Logged results export to valid, parsable JSON	Frames processed and logged	1. Call SaveToJson() 2. Deserialize output	Logged FrameResult list	File exists, deserializes without error, matches frame count	FR-05	Integration	Medium	Pass
TC-06	Constrained resource profile produces higher latency than baseline	Model loaded	1. Run under Baseline profile 2. Run under Constrained profile	bus.jpg, both profiles	Constrained latency measurably higher	FR-06, NFR-01	Non-functional	High	Pass
TC-07	System completes a full run under both resource profiles	Model + sample images available	1. Run batch under Baseline 2. Run batch under Constrained	bus.jpg, zidane.jpg	Both runs complete, 0 frames skipped	FR-07, NFR-07	Non-functional	High	Pass
TC-08	Summary report evaluates latency (NFR-01) pass/fail against threshold	Frames processed	1. Generate summary report	Logged FrameResults	NFR-01 result reflects accurate pass/fail vs 100ms threshold	FR-07, NFR-01	Non-functional	High	Pass
TC-09	Frame rate compliance (NFR-02) evaluated across larger batch	Model warmed up	1. Process 20-frame batch 2. Generate report	20 images (repeated samples)	NFR-02 result reflects real % of frames ≥ 15 FPS	FR-07, NFR-02	Non-functional	Medium	Fail (0% compliance, system achieves ~11 FPS avg)
TC-10	Cold-start latency measured separately from steady-state	Fresh detector instance	1. Measure first DetectFrame call 2. Process further frames	bus.jpg	Cold-start and steady-state latency reported separately	FR-07, NFR-09	Functional	Medium	Pass
TC-11	Batch processing completes under Baseline and HighLoad conditions	Model + sample images available	1. Run batch, Baseline load 2. Run batch, HighLoad	4 images	Both complete all frames, 0 skipped	FR-08	Negative	High	Pass
TC-12	Detector constructor throws clear error on missing model	No model file at given path	1. Construct OnnxYoloDetector with invalid path	Invalid model path	FileNotFoundException thrown with clear message	FR-09	Negative	High	Pass
TC-13	Batch run rejected before execution on missing input file	Model available	1. Call ProcessBatch with non-existent image path	Invalid image path	FileNotFoundException thrown, no frames processed	FR-11	Negative	High	Pass
TC-14	Corrupted frame is skipped and logged, run continues	Model available, 1 corrupted + 1 valid file	1. Process corrupted file alone 2. Process batch with corrupted + valid file	Corrupted file, bus.jpg	Corrupted frame marked skipped with reason; valid frame still processed	FR-10	Negative	High	Pass
TC-15	Empty batch request is rejected before execution	Model available	1. Call ProcessBatch with empty list	Empty list	ArgumentException thrown	FR-11	Boundary/Negative	Medium	Pass

Mohammed Yaacoub Abou Chlih 21138726

Matty Luriz 23206450

Github repository: <https://github.com/ExplosiveSalad/Quality-Assured-Vision.git>

Regression Testing

The full MSTest suite is re-run after every code change via Visual Studio's Test Explorer, following the regression practice after the Assessment 1 concurrency bug (where parallel test execution caused leakage between tests). [assembly: DoNotParallelize] stays to guarantee consistent, repeatable results across runs, and all 15 test cases are re-verified as part of this regression process before each commit.

Remaining Quality Risks

- TC-09's result raises a question about NFR-02's threshold, to be resolved in Task 8 rather than unresolved
- NFR-05 (configurability) is still validated manually rather than through automated testing (See Task 7), which holds a small risk of being under-documented compared to the automated suite.
- Test coverage relies on a small, fixed sample image set (2 base images, reused/repeated for volume), which lowers confidence in detection-accuracy claims across real-world conditions.

Task 4: Test Automation and Quality Engineering

- Identify suitable test cases for automation and justify the selection.
- Implement automated tests using an appropriate tool or framework where suitable.
- Report automated test execution results using logs, screenshots, reports, or repository evidence.
- Discuss the benefits, limitations, and maintainability of the automated test suite.
- Where automation is not appropriate, justify the use of manual or exploratory testing.

Task 5: Continuous Integration and DevOps QA

- Describe the development and integration workflow.
- Implement a simple CI workflow or equivalent automated quality check where appropriate.
- Define quality gates such as successful build, passing tests, linting, code review, or coverage threshold.
- Demonstrate smoke or sanity testing where relevant.
- If CI/CD is not technically appropriate, justify an alternative quality workflow.

Mohammed Yaacoub Abou Chlih 21138726

Matty Luriz 23206450

Github repository: <https://github.com/ExplosiveSalad/Quality-Assured-Vision.git>

Development and Integration Workflow

Development follows a feature-branch workflow: each team member works on a specific branch for their assigned tasks, committing regularly (minimum 2 commits/week) before merging into main via GitHub at the end. This branch-per-task structure, in place since Assessment 1, keeps individual contributions traceable and reviewable that supports the GitHub-evidence requirement for the contribution statement (Task 11).

CI Workflow Implementation

A GitHub Actions workflow was made to automatically build the solution and run the full MSTest suite on every push and pull request to main. This provides instant, automated feedback on whether a change breaks the build or any existing test, rather than relying solely on manual verification before merging.

(See `.github/workflows/ci.yml` for the file)

Quality Gates Defined

- **Successful build:** The workflow fails instantly if dotnet build fails, stopping any broken code from being considered mergeable
- **All tests passing:** dotnet test must exit properly; any of the 15 test case failures (aside from TC-09 NFR-02 result) would fail the workflow
- **Test result artifact retention:** .trx test result files are uploaded as build artifacts, giving a reviewable record of which tests passed/failed on each commit

Smoke/Sanity Testing

The CI workflow itself functions as a smoke test: a successful dotnet build asserts that the solution compiles with no missing references or syntax errors, and the full test run confirms the core detection pipeline (TC-01 through TC-15) stays functional after each change, catching regressions like the concurrency bug found in Assessment 1 before they caused further issues.

Mohammed Yaacoub Abou Chlih 21138726

Matty Luriz 23206450

Github repository: <https://github.com/ExplosiveSalad/Quality-Assured-Vision.git>

Task 6: Defect Management and Root Cause Analysis

- Maintain a final defect register covering defects found during development and testing.
- Define severity, priority, and defect status categories.
- Describe the defect lifecycle used in the project.
- Analyse at *least three (3)* significant defects using root cause analysis.
- Propose corrective actions and defect-prevention measures.
- Explain how defect analysis improved product or process quality.

Task 7: Quality Testing

- Conduct quality testing relevant to the project, such as security, performance, usability, accessibility, compatibility, reliability, or maintainability testing.
- Address at *least two (2)* quality areas where applicable.
- Use risk-based reasoning to justify the selected testing activities.
- Document testing evidence, results, limitations, and recommended improvements.

Task 8: Quality Metrics and Final Release Decision

- Present relevant quality metrics, such as test pass rate, defect severity distribution, requirement coverage, test coverage, automation coverage, code quality indicators, or commit activity.
- Evaluate the completed system against requirements, acceptance criteria, and quality attributes.
- Summarise testing results, defect status, metrics, limitations, and residual risks.
- State whether the system is ready for release, conditionally ready, or not ready for release.
- Justify the release decision using evidence.

Task 9: AI-Assisted Development and Critical Reflection

- Explain how GitHub Copilot was used for implementation, testing, debugging, refactoring, or QA activities.
- Provide representative evidence of AI-assisted development.

Mohammed Yaacoub Abou Chlih 21138726

Matty Luriz 23206450

Github repository: <https://github.com/ExplosiveSalad/Quality-Assured-Vision.git>

- Explain how AI-assisted outputs were reviewed, modified, tested, accepted, or rejected.
- Identify any quality risks found in AI-assisted outputs, such as incorrect logic, insecure code, poor maintainability, or unsuitable assumptions.
- Reflect on how AI-assisted development affected productivity, learning, testing, software quality, and professional practice.

Task 10: Quality Improvement and Future Recommendations

- Identify the main quality lessons learned from the project.
- Recommend improvements to the development and QA process.
- Propose future improvements to testing, automation, CI/CD, monitoring, security, usability, accessibility, or maintainability.
- Explain how the project could evolve toward stronger continuous quality engineering practice.

Task 11: Team Contribution and Professional Reflection

- Provide a contribution statement for each group member.
- Provide evidence from GitHub commits, project artefacts, testing work, documentation, and implementation tasks.
- Include a brief individual reflection from each group member.

Task 12: Project Presentation and Demonstration

Each group must prepare a 15-slide presentation and demonstrate the completed project. The presentation should clearly summarise the problem, final solution, quality assurance activities, testing evidence, and final quality evaluation. The presentation should cover:

- Project title, group members, problem, stakeholders, and scope
- Key functional and non-functional requirements
- Final prototype, implemented features, and changes after Assessment 1
- Testing approach, coverage, execution results, and key findings
- Automation, CI/CD, or equivalent quality workflow
- Defects, root cause analysis, and corrective actions
- Quality metrics, advanced testing, and release decision
- GitHub Copilot use, limitations, risks, and future improvements
- Individual contributions and lessons learned.

Mohammed Yaacoub Abou Chlih 21138726

Matty Luriz 23206450

Github repository: <https://github.com/ExplosiveSalad/Quality-Assured-Vision.git>

All students must participate in the demonstration and show the working prototype or quality improvement solution. The demo should include key features, important test cases, automated tests or CI results where applicable, defect fixes, and evidence that major requirements have been satisfied. Students must be prepared to answer questions about the project, code, testing, quality artefacts, AI-assisted development, and individual contributions.

Deliverables

- Final Project and Software Quality Assurance Report: *6000--7000 words*
- Source code and GitHub repository link; provide evidence from GitHub commits.
- Contribution statement for each group member
- Project presentation
- Turnitin Similarity Report